

КУРСОВА РОБОТА · 2026

Веб-додаток для керування задачами

Node.js · Angular · MongoDB

Стець Т.В. · Група ПМА-31

Керівник: Голуб Б.М.

01

Постановка задачі

Проблема

Команди витрачають час на неефективне відстеження задач. Потрібна зручна система з ролями, аналітикою та спільним доступом.

Мета роботи

Спроектувати та реалізувати SPA-веб-додаток для керування задачами з використанням Node.js, Angular та MongoDB.

Завдання

- Аналіз архітектури клієнт-сервер та SPA-підходу
- Порівняння стеку з альтернативами (PHP, Django, Spring)
- Реалізація серверного REST API на Node.js + Express
- Побудова Angular SPA з JWT-авторизацією та ролями

02 / Технологічний стек

Node.js

Сервер

Асинхронна модель
Event Loop

V8-рушій, висока
продуктивність

Express.js, npm-
екосистема

Angular

Клієнт

SPA без
перезавантаження

TypeScript,
компонентний підхід

Guards, Router,
HttpClient

MongoDB

База даних

NoSQL документи,
гнучка схема

Mongoose ODM
для Node.js

Колекції users
та tasks

JWT

Безпека

Stateless-токени
авторизації

middleware
authMiddleware

Ролі: user /
admin

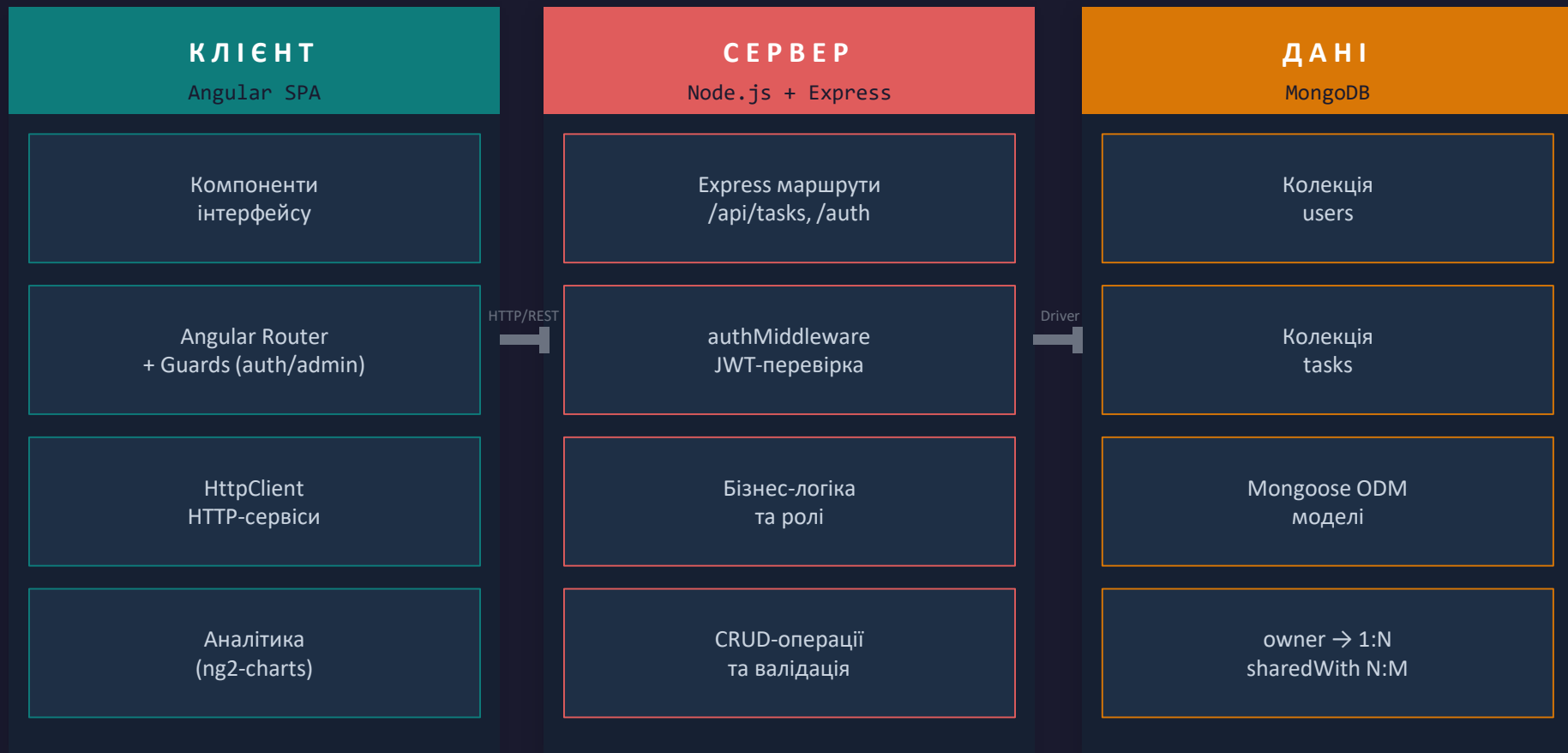
03 / Порівняння з альтернативами

Стек Node.js + Angular обрано як оптимальний для SPA-системи з єдиною мовою та REST API

Критерій	Node.js + Angular	PHP	Django (Python)	Spring (Java)
Швидкість розробки	Висока	Висока	Середня	Низька
SPA / клієнтська частина	Вбудований Angular	Стороннє	Стороннє	Стороннє
Єдина мова (JS/TS)	✓ Так	✗ Ні	✗ Ні	✗ Ні
Масштабованість	Висока	Середня	Висока	Дуже висока
Поріг входження	Середній	Низький	Середній	Високий

★ Node.js + Angular — вибір для даного проєкту

04 / Архітектура системи



05

Що реалізовано

- JWT-авторизація + 2 ролі
- CRUD задач (personal / shared)
- sharedWith — спільний доступ
- Аналітика: графіки ng2-charts
- Адмін-панель
- Темний режим + мультимова

REST API маршрути

POST	/auth/register	— реєстрація
POST	/auth/login	— вхід, JWT-токен
GET	/api/tasks	— список задач
POST	/api/tasks	— нова задача
PUT	/api/tasks/:id	— редагування
DEL	/api/tasks/:id	— видалення

Mongoose-моделі: User · Task

06 / Тестування системи

1



Вхід з правильними даними → сторінка задач

2



Невірний пароль → помилка авторизації 401

3



Створення задачі → з'являється у списку

4



Редагування → зміни зберігаються у MongoDB

5



Видалення → задача зникає зі списку

6



Запит без токена → сервер повертає 401

7



user не бачить чужі задачі

8



admin бачить усі задачі системи

9



Аналітика — графіки будуються коректно

9 / 9 тестів успішно · 100% проходження · Усі ролі та права підтверджено

Висновки

Реалізовано —	повноцінний SPA-веб-додаток: авторизація, CRUD задач, спільний доступ, аналітика, адмін-панель
Архітектура —	тришарова: Angular SPA → Node.js + Express → MongoDB — забезпечує масштабованість
Безпека —	JWT + authMiddleware + система ролей (user / admin) + guard-захист маршрутів
Стек обґрунтовано —	Node.js + Angular є оптимальним для SPA: єдина мова, висока швидкість розробки
Тестування —	9 / 9 сценаріїв пройдено успішно, функціонал підтверджено